

```
In [1]: import os
import numpy as np
from PIL import Image
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import accuracy_score, classification_report
```

```
In [2]: DATASET_DIR = "../planetsdataset"

TRAIN_DIR = os.path.join(DATASET_DIR, "training")
VALID_DIR = os.path.join(DATASET_DIR, "validation")
TEST_DIR = os.path.join(DATASET_DIR, "test")

CLASS_NAMES = ["earth", "asteroid"]
```

```
In [3]: def load_images_from_split(split_dir, class_names, img_size = (256, 144)):
    X = []
    y = []

    for label, class_name in enumerate(class_names):
        class_dir = os.path.join(split_dir, class_name)

        if not os.path.isdir(class_dir):
            continue

        for file_name in os.listdir(class_dir):
            file_path = os.path.join(class_dir, file_name)

            try:
                img = Image.open(file_path).convert("RGB")
                img = img.resize(img_size)
                img_array = np.array(img, dtype=np.float32) / 255.0
                X.append(img_array.flatten())
                y.append(label)
            except Exception as e:
                print(f"skipping {file_path}: {e}")

    return np.array(X), np.array(y)
```

```
In [4]: IMG_SIZE = (64, 64)
X_train, y_train = load_images_from_split(TRAIN_DIR, CLASS_NAMES, IMG_SIZE)
X_valid, y_valid = load_images_from_split(VALID_DIR, CLASS_NAMES, IMG_SIZE)
X_test, y_test = load_images_from_split(TEST_DIR, CLASS_NAMES, IMG_SIZE)

print("Train:", X_train.shape, y_train.shape)
print("Validation:", X_valid.shape, y_valid.shape)
print("Test:", X_test.shape, y_test.shape)
```

```
Train: (421, 12288) (421,)
Validation: (101, 12288) (101,)
Test: (58, 12288) (58,)
```

```
In [5]: # train model
model = LogisticRegression(
    max_iter=200,
```

```

    random_state=42,
    n_jobs=-1
)

model.fit(X_train, y_train)

```

Out[5]:  **LogisticRegression**
 LogisticRegression(max_iter=200, n_jobs=-1, random_state=42)

```

In [6]: # validation
y_valid_pred = model.predict(X_valid)
print("Validation Accuracy:", accuracy_score(y_valid, y_valid_pred))
print("Validation Report:")
print(classification_report(y_valid, y_valid_pred, target_names=CLASS_NAMES))

```

Validation Accuracy: 0.9405940594059405

Validation Report:

	precision	recall	f1-score	support
earth	0.92	0.98	0.95	59
asteroid	0.97	0.88	0.93	42
accuracy			0.94	101
macro avg	0.95	0.93	0.94	101
weighted avg	0.94	0.94	0.94	101

```

In [7]: y_test_pred = model.predict(X_test)
print("Test Accuracy:", accuracy_score(y_test, y_test_pred))
print("Test Report:")
print(classification_report(y_test, y_test_pred, target_names=CLASS_NAMES))

```

Test Accuracy: 0.896551724137931

Test Report:

	precision	recall	f1-score	support
earth	0.93	0.86	0.89	29
asteroid	0.87	0.93	0.90	29
accuracy			0.90	58
macro avg	0.90	0.90	0.90	58
weighted avg	0.90	0.90	0.90	58

In []:

In []: